

Base36, Bijective Numeration, and Symbolic Character Encoding in OMI

A Whitepaper on the $W = 36$ Orbit Alphabet, Hidden Five Root, and Unicode Symbolic Projection Layer

Status: Conceptual Whitepaper / v1.0.0-RC1 Alignment Draft

Scope: Base36 relevance, Delta Law period recovery, bijective numeration, unary root logic, signed/folded symbolic digits, emoji/domino projection, and OMI symbolic encoding

Core Claim: Base36 is relevant to OMI because the Delta Law derives an orbital tracking width of $W = 36$. Base36 supplies a compact printable alphabet for that width, while bijective/unary/signed numeration explain how OMI can separate hidden roots, visible steps, complement folds, and symbolic carriers.

Abstract

OMI is an address-centered protocol model. Its core wire object is the 128-bit pointer:

```
omi-S0-S1-S2-S3-S4-S5-S6-S7/prefix
```

The pointer remains the authority. Symbolic encodings such as Base36, emoji, domino tiles, Code16K, JABCode, and visual glyph systems do not replace the pointer. They project the pointer.

The relevance of Base36 comes from the Delta Law:

$$\Delta_C(x) = \text{rotl}(x,1) \oplus \text{rotl}(x,3) \oplus \text{rotr}(x,2) \oplus C$$

The law produces a period-8 orbit on the 16-bit space. Period 8 points to prime 73 as the smallest prime whose decimal expansion repeats with period 8:

$$1 / 73 = 0.01369863 \ 01369863 \ \dots$$

The repeating block is:

$$B = [0, 1, 3, 6, 9, 8, 6, 3]$$

The digit sum is:

$$W = 0 + 1 + 3 + 6 + 9 + 8 + 6 + 3 = 36$$

Thus OMI receives a natural 36-wide tracker:

```
divmod(position, 36)
```

Base36 supplies exactly 36 printable symbols:

0-9 + A-Z

Therefore:

Base36 = visible alphabet for $W = 36$

Base36 is not the law. Base36 is the symbolic projection alphabet for the law-derived orbit width.

1. The Governing Hierarchy

The correct order is:

```
Delta Law
→ period 8
→ prime 73
→ block B = 01369863
→  $W = 36$ 
→ divmod(position, 36)
→ Base36 symbolic alphabet
→ emoji/domino/visual projection
```

The symbolic encoding layer must never be placed before the Delta Law.

Wrong:

Base36 creates the OMI orbit.

Correct:

The Delta Law creates $W = 36$.
Base36 names the 36 positions.

This distinction preserves the authority boundary:

law first
symbol second
projection third

2. Why Base36 Fits OMI

Base36 has 36 symbols:

0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ

OMI's orbit tracker has width 36:

$W = 36$

So every Base36 character can represent exactly one local orbit remainder:

$\text{char}_{36} \in \{0..9, A..Z\}$
 $\text{value}_{36} \in \{0..35\}$

This gives a printable symbolic layer for:

remainder36
local240 substep
Karnaugh-torus coordinate
canvas cell
symbolic route
barcode character
emoji bridge
DOM/CSSOM metadata

The core mapping is:

```
value = parseBase36(char)
quotient, remainder = divmod(position, 36)
```

The character itself is not the source of truth. It is the readable name of a position inside the 36-wide tracker.

3. Base36 as the Orbit Alphabet

The period block from `1/73` is:

```
01369863
```

These are stepping digits, not the full alphabet.

The full local symbolic alphabet is Base36:

```
0 1 2 3 4 5 6 7 8 9 A B C ... Z
```

OMI should distinguish:

```
period digits = active recurrence guide
Base36 symbols = complete 36-position naming alphabet
```

The period digits are:

```
{0,1,3,6,8,9}
```

The visible stepping digits omit `5`.

That omission matters because the factor `5` is not a visible step. It is hidden in the five-fold root:

```
5! = 120
240 = 2 × 5!
```

So Base36 names the full tracker, but the active Delta/73 stepping path walks only the recurrence residues.

4. The Hidden Five and the Visible Base36 Alphabet

A subtle point:

Base36 contains the character `5`.

The Delta stepping digits from `1/73` do not.

This is not a contradiction.

Base36 is the full alphabet:

```
all positions 0..35
```

The Delta stepping sequence is the orbit trace:

```
0,1,3,6,9,8,6,3
```

The factor `5` remains hidden as the packet root:

```
5! = 120
```

The symbol `5` may exist as a printable character in Base36, but it should not be interpreted as the root factor itself.

Therefore:

```
Base36 digit "5" = symbolic position 5 in the visible alphabet  
factor 5 = hidden ontological root inside 5!
```

These must not be confused.

Canonical rule:

```
The character "5" may appear in Base36 encoding.  
The factor 5 does not appear as an operational stepping digit in the 1/73  
carrier.
```

That preserves the hidden-root doctrine:

The climb does not step to 5.
The climb preserves 5.

5. Bijective Numeration and OMI

Bijective numeration matters because OMI wants one-to-one symbolic names for states.

In ordinary positional notation, leading zeros can create multiple strings for the same value:

1
01
001

All can represent the same number in ordinary notation.

That is bad for canonical addressing because OMI needs stable references.

Bijective numeration solves this by using digits without a zero digit. In bijective base-k, the digit set is:

$\{1, 2, \dots, k\}$

Zero is represented by the empty string.

That matters for OMI because:

empty string = null / identity / root absence
nonempty string = active symbolic state

This maps cleanly to the hidden-root model:

empty = presupposed identity
visible digit = projected state

OMI can use ordinary Base36 for compatibility, but it should also define a **bijective Base36 mode** for canonical symbolic state names.

6. Ordinary Base36 vs Bijective Base36

Ordinary Base36 uses:

```
0 1 2 3 4 5 6 7 8 9 A ... Z
```

Bijective Base36 would use 36 nonzero symbols.

One possible OMI bijective alphabet is:

```
1 2 3 4 5 6 7 8 9 A B C ... Z Ω
```

where Ω stands for the 36th digit.

But OMI should be careful. Adding a new symbol like Ω may harm URL safety and DOM selector simplicity.

A practical OMI-compatible bijective Base36 alphabet could instead be:

```
1 2 3 4 5 6 7 8 9 A B C ... Z _
```

where $_$ is the 36th digit.

However, for browser and CSS selector safety, the canonical rule should be:

```
Ordinary Base36 is the transport alphabet.  
Bijective Base36 is the canonical no-leading-zero state naming discipline.
```

That means:

```
Base36 carries values.  
Bijective discipline controls uniqueness.
```

7. Unary Numeration and the Root Layer

Unary is relevant because it is the simplest bijective system.

Unary uses one mark repeatedly:

```
|  
||  
|||  
||||
```

In OMI, unary corresponds to repeated presence, not positional value.

This matters for roots.

A root is not stepped by changing digit value. A root is established by presence.

Thus unary maps to:

```
root assertion  
presence count  
tick count  
proof repetition  
witness accumulation
```

The five-fold root can be understood as a unary-style presupposition:

```
5! is not a digit step.  
5! is an asserted root.
```

Unary is therefore the correct analogy for the hidden packet root:

```
unary = presence-based root encoding  
Base36 = visible orbit alphabet
```

In OMI:

```
unary proves presence  
Base36 names position  
bijective discipline gives unique state names  
signed digits define folds
```

8. Signed-Digit Representation and Fold Surfaces

Signed-digit systems are relevant because they let digits carry positive and negative values.

OMI has the same need.

Every visible state has a complement or failure surface:

```
valid ↔ invalid  
source ↔ shadow  
presence ↔ absence  
direct ↔ folded  
left ↔ right  
positive ↔ negative
```

In OMI origami terms:

```
ones-complement = mirror fold  
two's-complement = signed inversion fold
```

Signed symbolic digits give OMI a language for representing fold direction.

A signed Base36 mode could interpret a symbol as:

```
(+ symbol) direct state  
(- symbol) folded/complement state
```

But OMI should avoid raw plus/minus prefixes inside production pointer grammar. Instead, fold state should be encoded by address fields, CSS classes, or data attributes:

```
data-omi-fold="direct"  
data-omi-fold="complement"
```

or by a rule-bound symbolic carrier:

```
fold = 0 → direct  
fold = 1 → complement
```

This preserves the OMI rule:

```
No symbolic character overrides pointer validity.
```

9. The Three Remembered Numeration Ideas

The relevant numeration ideas are likely:

1. Bijective numeration
2. Unary / base-one numeration
3. Signed-digit representation

A fourth related idea is also important:

4. Mixed-radix or non-power positional numeration

These map to OMI as follows:

Numeration idea	OMI meaning
Bijective numeration	unique symbolic state naming
Unary	root presence / repeated assertion / hidden root
Signed-digit representation	complement, fold, failure, inverse state
Mixed-radix / non-power positional	factorial tower, 720/5040 cycles, base-60 scheduler

This gives OMI a full symbolic numeration stack:

```
unary root
→ bijective state name
→ signed fold
→ mixed-radix tower
→ Base36 visible orbit alphabet
```

10. Mixed-Radix and Factorial Towers

OMI does not use one uniform radix everywhere.

It uses several meaningful widths:

- 2 complement / orientation
- 3 S-P-O semantic role

```
4  FS/GS/RS/US selector face
7  Fano selector
8  Delta period / address rail
12 base-60 clock divisor class
36 orbit tracker width
240 active byte bridge
720 semantic sweep
5040 replay ring
```

This is naturally a mixed-radix system.

The canonical replay decomposition is:

```
slot5040 = fano7 × 720 + role3 × 240 + local240
```

with:

```
fano7    ∈ 0..6
role3    ∈ 0..2
local240 ∈ 0..239
```

Base36 appears inside this as a local symbolic address for orbit tracking:

```
local remainder = position mod 36
Base36 char = name(remainder)
```

So OMI does not replace the factorial tower with Base36. It uses Base36 as the visible language for one of the tower's key tracker widths.

11. Base36 and the 240-State Bridge

The 240 bridge is:

```
240 = 2 × 5!
240 = 15 × 16
240 = 6! / 3
```

The `W = 36` tracker sits inside local240 as an orbit addressing layer.

One useful decomposition is:

$$\text{local240} = q_{36} \times 36 + r_{36}$$

where:

$$r_{36} \in 0..35$$

But 240 is not divisible by 36 evenly:

$$240 = 6 \times 36 + 24$$

This is useful, not a problem.

It means local240 contains:

6 full Base36 bands
+ 24 selector residues

The leftover 24 is exactly:

$$4! = 24$$

This is a powerful bridge.

So:

$$240 = 6 \times 36 + 24$$

can be read as:

six full orbit alphabets
+ one four-fold selector face

This gives Base36 direct relevance to the four-fold visible selector.

Canonical statement:

The 240-state bridge contains six complete Base36 orbit bands plus one 4! selector residue.

This is one of the strongest reasons Base36 belongs in OMI.

12. Base36, 4!, and the Remainder Face

The identity:

$$240 = 6 \times 36 + 24$$

means:

```
local240 divmod 36
→ quotient ∈ 0..6
→ remainder ∈ 0..35
```

The quotient has seven possible values for local240 because values 216..239 form the last partial band:

0..35	band 0
36..71	band 1
72..107	band 2
108..143	band 3
144..179	band 4
180..215	band 5
216..239	partial band 6 with 24 values

The last partial band has 24 values:

$$216..239 = 24 \text{ states}$$

That is the four-fold selector surface:

$$4! = 24$$

Thus the local240 bridge naturally decomposes into:

six full Base36 bands
+ one 24-state selector cap

This matches OMI's rule/fact model:

Base36 bands = operational orbit naming
24-state cap = projection selector face

13. Base36 and the Hidden Five

Recall:

$$240 = 2 \times 5!$$

and:

$$240 = 6 \times 36 + 24$$

So the same bridge can be read two ways:

Hidden-root reading:

$$240 = 2 \times 5!$$

Visible-symbol reading:

$$240 = 6 \times 36 + 4!$$

This is the most important result.

It says:

240 has a hidden semantic root and a visible symbolic decomposition.

The hidden side is:

5! packet root × 2 orientation

The visible side is:

six Base36 orbit bands + four-fold selector cap

So the full bridge identity is:

$$2 \times 5! = 6 \times 36 + 4!$$

Check:

$$\begin{aligned} 2 \times 120 &= 240 \\ 6 \times 36 + 24 &= 216 + 24 = 240 \end{aligned}$$

Therefore:

$$2 \times 5! = 6 \times 36 + 4!$$

This is the canonical Base36 bridge law.

14. OMI Interpretation of the Base36 Bridge Law

The equation:

$$2 \times 5! = 6 \times 36 + 4!$$

means:

hidden packet root × orientation
=
six visible orbit alphabets + selector face

In OMI language:

```
hidden five-fold root
+ complement orientation
=
Base36 visible stepping bands
+ four-fold projection face
```

This exactly matches the hidden-root doctrine:

```
5-fold = hidden root
4-fold = visible face
240 = bridge
Base36 = symbolic orbit alphabet
```

This is a core result.

It should be added to `POSTULATES.md`, `DECLARATIONS.md`, or the object model docs.

15. Base36 Character Classes in OMI

OMI should partition Base36 characters into semantic bands.

A practical partition:

```
0-9 = numeric residues / direct visible steps
A-F = hex-compatible bridge states
G-L = geometry / polygon states
M-R = memory / replay states
S-X = semantic / S-P-O states
Y-Z = reserved / escape / provider envelope
```

This is convenient but arbitrary unless grounded by tests.

A more OMI-native partition is derived from the period block:

```
Active period digits: 0,1,3,6,8,9
Silent visible digits: 2,4,5,7
Alphabetic extension: A-Z
```

This gives three classes:

period-active
period-silent
extension

Where:

period-active = directly used by 1/73 stepping
period-silent = present in Base36 but not active in the period block
extension = higher symbolic orientation and projection

The digit 5 belongs to period-silent.

This is important:

Base36 contains 5.
The Delta stepping block does not use 5.
So Base36 digit 5 is a visible symbol, not the hidden root factor.

16. Symbolic Character Encoding Stack

The symbolic encoding stack should be:

Base36
→ emoji
→ domino
→ barcode
→ DOM/CSSOM

Each layer has a role:

Layer	Role
Base36	W=36 orbit alphabet
Emoji	S-P-O semantic carrier
Domino tiles	two-node pair / car-cdr tile
Code16K	stacked linear page header
JABCode	colored matrix body

Layer	Role
DOM/CSSOM	browser-visible projection surface

Canonical boundary:

Symbols project OMI state.
They do not define OMI state.

17. Emoji as Semantic Carrier

Emoji are useful because they are non-alphanumeric symbolic code points with semantic categories.

OMI can map emoji into:

Subject → point / noun / object symbol
Predicate → line / action / operator symbol
Object → face / target / domain symbol

The runtime should validate emoji using exact Unicode data, not hand-picked block ranges.

The correct route is:

emoji code point or sequence
→ Unicode validation
→ CLDR group/subgroup
→ WordNet proxy lattice
→ S-P-O triple
→ OMI pointer
→ projection

Emoji should be treated as semantic carriers over the OMI state, not as authority.

18. Domino Tiles as Pair Encoding

Unicode domino tiles are useful because they visually encode a pair:

```
pipA | pipB
```

OMI can read this as:

```
car | cdr  
source | target  
subject | object  
x | y  
top | bottom  
left | right
```

Domino tiles are therefore a natural symbolic carrier for pairwise relations.

A canonical domino route would be:

```
domino code point  
→ orientation  
→ pip pair  
→ pair route  
→ local240  
→ slot5040  
→ CSSOM projection
```

Any exact claim about the Unicode domino block size or codepoint ranges should be tested before being canonical.

19. The 16-Symbol Register

The 16-emoji or 16-character register is useful because it mirrors binary16:

```
1 sign  
5 exponent  
10 stored significand  
+ hidden lead = 11 precision
```

In OMI symbolic form:

```
1 sign symbol  
5 exponent symbols
```

10 payload symbols
+ hidden lead rule

This maps to:

sign symbol → fold / polarity
5 exponent symbols → $2^5 = 32$ omi--- body
11 precision → witness / prime signature / hidden lead

This is a visible register model for the 32/64 split:

$2^5 = 32$ body
 $2^6 = 64$ folded truth-table envelope

The hidden lead rule also reinforces the hidden-root doctrine:

visible payload + hidden presupposed bit = full precision
visible climb + hidden 5! root = full OMI packet identity

20. Proposed Rules

Rule: Base36 Orbit Alphabet

```
# [Rule 0x72]: Base36 Orbital Symbol Alphabet
#   Base36 symbols MUST project the W=36 orbit tracker and MUST NOT replace
#   the Delta Law as the source of orbital state.
omi-0000-0000-0000-0000-0000-0000-0036-0072/128 MUST project-base36-orbital-
symbols
```

Rule: Bijective Symbol Canonicalization

```
# [Rule 0x73]: Bijective Symbol Canonicalization
#   Canonical symbolic state names SHOULD avoid leading-zero ambiguity. When
#   bijective mode is active, zero is represented by empty state, not by a
#   digit.
omi-0000-0000-0000-0000-0000-0000-0036-0073/128 MUST canonicalize-bijective-
symbol-state
```

Rule: Unary Root Presence

```
# [Rule 0x74]: Unary Root Presence
#   Unary-style symbolic repetition MAY assert root presence, but MUST NOT
#   replace five-fold packet-root validation.
omi-0000-0000-0000-0000-0000-0000-0001-0074/128 MUST preserve-unary-root-
presence-boundary
```

Rule: Signed Fold Symbol Boundary

```
# [Rule 0x75]: Signed Fold Symbol Boundary
#   Signed symbolic forms MAY project direct/complement orientation, but MUST
#   NOT override Q(S), Delta orbit, or RULES.omi / FACTS.omi declarations.
omi-0000-0000-0000-0000-0000-0000-0002-0075/128 MUST preserve-signed-fold-
symbol-boundary
```

Rule: Unicode Symbolic Carrier Authority

```
# [Rule 0x76]: Unicode Symbolic Carrier Authority
#   Emoji, domino, and barcode symbols MUST act as carriers over validated OMI
#   state, not as authorities that redefine pointer validity.
omi-0000-0000-0000-0000-0000-0000-0000-0076/128 MUST preserve-unicode-symbolic-
carrier-authority
```

21. Proposed Facts

```
# --- Symbolic Encoding Facts ---

omi-0000-0000-0000-0000-0000-0000-0036-1001/128 FACT base36-orbit-alphabet-
active
omi-0000-0000-0000-0000-0000-0000-0036-1002/128 FACT base36-width-equals-delta-
orbit-tracker
omi-0000-0000-0000-0000-0000-0000-0036-1003/128 FACT base36-visible-five-is-not-
hidden-factor-five

omi-0000-0000-0000-0000-0000-0000-0001-1001/128 FACT unary-root-presence-model-
documented
omi-0000-0000-0000-0000-0000-0000-0002-1001/128 FACT signed-fold-symbol-model-
documented
```

```
omi-0000-0000-0000-0000-0000-0000-0073-1001/128 FACT bijective-symbol-  
canonicalization-documented  
omi-0000-0000-0000-0000-0000-0000-0076-1001/128 FACT unicode-symbols-are-  
carriers-not-authorities
```

22. Implementation Guidance

The symbolic compiler should expose a strict boundary:

```
function projectBase36Symbol(char) {  
  const value = parseInt(char, 36);  
  if (!Number.isInteger(value) || value < 0 || value >= 36) {  
    return { valid: false, reason: "not-base36" };  
  }  
  
  return {  
    valid: true,  
    symbol: char.toUpperCase(),  
    value36: value,  
    role: classifyBase36Symbol(value),  
  };  
}
```

Then OMI state should derive from the law, not the symbol:

```
function projectOrbitPosition(position) {  
  const q = Math.floor(position / 36);  
  const r = position % 36;  
  
  return {  
    quotient: q,  
    remainder36: r,  
    symbol: r.toString(36).toUpperCase(),  
  };  
}
```

The authority boundary:

```
const projection = projectOrbitPosition(position);  
const symbol = projection.symbol;
```

not:

```
const position = parseSymbolAsLaw(symbol);
```

Symbols read the state. They do not create the state.

23. Canonical Formulas

The formulas that should become canonical are:

$$\Delta_C(x) = \text{rotl}(x,1) \oplus \text{rotl}(x,3) \oplus \text{rotr}(x,2) \oplus C$$

$1/73 = 0.01369863\dots$
 $B = [0,1,3,6,9,8,6,3]$
 $W = \text{sum}(B) = 36$

Base36 = visible alphabet for $W = 36$

$$240 = 2 \times 5! = 15 \times 16 = 6!/3$$
$$240 = 6 \times 36 + 4!$$
$$2 \times 5! = 6 \times 36 + 4!$$
$$\text{slot5040} = \text{fano7} \times 720 + \text{role3} \times 240 + \text{local240}$$

These give the complete bridge:

hidden five-fold root
→ 240 bridge
→ Base36 orbit alphabet
→ four-fold selector cap
→ S-P-O sweep
→ Fano replay ring

24. Final Position

Base36 matters because it is not arbitrary.

It exactly matches the Delta-derived width:

$$W = 36$$

Bijjective numeration matters because OMI needs unique symbolic state names.

Unary matters because OMI roots are presence assertions before they are visible steps.

Signed-digit representation matters because OMI has fold, complement, and failure surfaces.

Mixed-radix matters because OMI's real runtime is not one base; it is a tower:

2, 3, 4, 7, 8, 12, 36, 240, 720, 5040

The symbolic encoding layer should therefore be understood as:

```
unary root
→ bijective canonical name
→ signed fold
→ Base36 orbit alphabet
→ Unicode semantic carrier
→ barcode / DOM / CSSOM projection
```

Base36 is the first clean visible alphabet for this stack because it gives exactly one printable symbol per orbit-tracker residue.

25. One-Sentence Summary

Base36 is relevant to OMI because the Delta Law derives a 36-wide orbit tracker from the period-8 1/73 block; bijective numeration gives canonical no-ambiguity symbolic names, unary explains hidden root presence, signed-digit representation explains complement/fold surfaces, and together these let Base36, emoji, domino, barcode, and DOM/CSSOM glyphs act as visible carriers of OMI state without replacing the authority of the 128-bit pointer, Q(S), Delta orbit, RULES.omi, and FACTS.omi.